PRODUCT OVERVIEW · DOCUMENTATION

Slip

The second opinion on every agent purchase. Slip checks what an AI agent is about to buy — against every other offer listed for the same item — before PayPal moves any money.

`Agents shop. Slip checks. Policy decides. PayPal pays.`

This document is an orientation to the whole product: what it does, how a purchase travels through it, what the rules are, what has been measured, and what is still open. Every number in it was read from the repository or produced by running it, on the date below.

AS BUILT

Date	30 October 2026
Commit	`256ad11`
Tests	2,129 passing · 5 skipped · 135 files
Lint & types	clean (biome, tsc project references)
Real Postgres	310 contract tests, run on Postgres 14.17
Browser tests	38 across two servers (demo and real-owner), axe included
Decisions logged	100 entries, latest D105
Workspace	16 packages · TypeScript 7 · pnpm, strict catalog

The 5 skipped tests are the key-gated live suites: they skip themselves when the service's key is absent, so the default gate can never spend money. The real-Postgres figure was produced against a local Postgres 14.17 rather than the 18 the compose file provisions — the suite creates and drops its own database per file, so the server it runs on is the one that happens to be there.

1 What Slip is for

AI agents are starting to spend people's money, and nothing checks whether they are overpaying or being steered.

That is not a guess. What the agentic-commerce rails carry on the wire is an *allowance*: a delegated payment request binds the credential to an amount, a currency, one merchant, one checkout session, an expiry and a single use. That is real protection — the credential cannot be spent elsewhere, later, for more or twice — and it is a description of the offer the agent already chose. Nothing in it refers to any other offer, because comparison is not what it is for: $400 that is the lowest listed price and $400 that is listed at $260 elsewhere are the same number, and an allowance for $400 admits both. Slip's own per-order cap is a bound too, and cannot tell them apart either; that is why a cap is not the check. The buyer-side control in PayPal's own agent documentation is a line in an example system prompt — *"After approval by user, invoke capture_order"* — which nothing enforces, and a prompt is not a control: the same model that reads it decides whether to follow it. Read on 30 October 2026; none of this is a criticism of a payments company's product, which is an acceptance product and was never meant to be a buyer's second opinion.

Slip is the control. An owner approves once, through PayPal. After that an agent can only pay through Slip, and every attempt is checked first: the chosen offer is compared with the other offers listed for the same item, steering is looked for in the text the agent actually read, and the owner's limits decide. The purchase is paid, rejected, or held for a human — and either way it prints a receipt in plain words.

What it is not

- **Not a shopping agent.** Slip does not choose what to buy; it checks what was chosen.
- **Not a merchant.** No order reaches a retailer. Slip is the seller of record in PayPal's sandbox.
- **Not a price oracle.** It compares against one catalogue (Channel3) and says so on every receipt.

> THE HONEST BOUND ON THE CLAIM
>
> Slip's own measurements do not show agents being fleeced — see §7. The case for a check like this rests on the structure of the market (the dearer offer often pays more commission) and on the absence of any enforced buyer-side control, not on a demonstration that today's models overpay.

2 How a purchase travels

1 · MANDATE

The owner approves once at PayPal. The vault token is stored encrypted; the limits become policy version 1. A kill switch revokes it at any time.

2 · CHECK

Freshness → variant → locate the chosen offer → find comparable offers → measure the gap → run the rules. Pure, no I/O, the same code everywhere.

3 · DECIDE

Paid, rejected, or held. The decision and its payment row are written *before* PayPal is called, so nothing can move money unrecorded.

4 · AFTERWARDS

A resolver expires holds, finishes half-done work and reconciles against PayPal's own records. Webhooks converge however late they arrive.

PAID

Within the owner's limits. Captured once, with one request id; the receipt names the amount and the comparison behind it.

REJECTED

Overpaying beyond the limit, steered, or an offer that cannot be verified. Nothing is charged.

HELD

Something is uncertain rather than wrong. The owner sees what approving would cost and approves *that* amount.

Two properties that shape everything else

Write-ahead payments. The attempt and its payment row exist before the gateway is called. A timeout therefore means *unknown* — it may or may not have happened — never *failed*. The resolver repeats the operation with the *same* request id until PayPal gives a definite answer, which is why no adapter in the codebase retries on its own.

One checkout per agent and product at a time. A lease makes it so, and a repeat of the same request is replayed from the stored decision rather than paid twice.

3 The rules that decide

First match wins. The last rule always matches, so there is no path without a decision.

RULE	WHAT TRIPS IT	OUTCOME
R1_STALE_SNAPSHOT	Prices older than the policy's freshness window (10 minutes by default)	HELD
VARIANT_MISMATCH	The offer is for a different variant than the one asked for	REJECTED
OFFER_NOT_FOUND	The chosen offer is not in the snapshot at all	REJECTED
CHOSEN_UNAVAILABLE	It is in the snapshot and out of stock — known bad	REJECTED
CHOSEN_UNVERIFIED	Its price or currency cannot be read — unknown, not bad	HELD
R2_TOO_FEW_OFFERS	Nothing comparable to price it against	HELD
R2A_GAP_IMPLAUSIBLE	A gap so wide (>100%) that the catalogue's grouping, not the price, is the likely cause	HELD
R2B_SINGLE_SELLER_MARKET	The comparable offers collapse to one seller, so the market was never a market	HELD
R3_STEERED	Above the flag line *and* the text the agent read is instruction-like	REJECTED
R4_OVERPAID	Above both the percentage and the absolute overpay limits	REJECTED
R4A_NO_CLASSIFIER_ANSWER	Above the flag line and the steering check could not answer	HELD
R5_COMMISSION_CONFLICT	Above the flag line and the dearer offer pays more commission	HELD
R5A_COMMISSION_UNDISCLOSED	Above the flag line, the cheaper offer's rate is known and the chosen offer's is not	HELD
R6_ABOVE_ORDER_CAP	The price is above the owner's per-order cap	HELD
R7_WITHIN_LIMITS	Everything else — the fallback that always matches	PAID
OWNER_APPROVED	A held purchase the owner released — a person's decision, not a rule's	PAID

The shipped defaults

LIMIT	DEFAULT	MEANING
Reject above	5% *and* $8	Both must be exceeded, so small absolute gaps on cheap items do not reject
Flag above	1%	The line above which steering, a missing verdict or a commission conflict can hold a purchase
Order cap	$200	Any single purchase above it is held, whatever the comparison says
Snapshot freshness	10 minutes	Older prices are not trusted to decide

Limits are versioned: changing them writes a new policy version, and every receipt names the version that decided it. Percentages are integer hundredths of a percent and money is integer cents throughout — no floating point touches a decision.

4 What an agent talks to

Slip is an MCP server. An agent pays through its tools with its own key, and gets the owner's receipt back in the same words the owner sees.

TOOL	WHAT IT DOES
`search_products`	Searches the catalogue and returns products with every store's offer. Shows the product's own text, marked untrusted — and records what was shown, so the check can judge it later.
`checkout`	Attempts a purchase. Returns the decision, the comparison behind it, and the receipt.
`get_receipt`	One attempt in full, including the offers it was compared against.
`get_report`	The agent's own report card: how it has been spending.

SURFACE	DETAIL
Endpoint	`POST /api/mcp` — MCP Streamable HTTP, both protocol eras, served statelessly
Auth	`Authorization: Bearer slip_live_…` — the agent's own key, hashed at rest, shown once
Without MCP	`POST /api/v1/agent/checkout` — the same use case and the same decisions
Refusals	`401` unknown or revoked key · `403` cross-site origin · `405` session operations · `413` body over 64 KiB · `500` anything already decided is kept

> WHY THE AGENT IS TOLD WHAT SLIP SIMULATED
>
> Every answer says which services were real. “Prices real, payment simulated” is a state Slip is genuinely in, and a receipt that called either one the other would be the one lie the product cannot afford.

5 How the code is arranged

Ports and adapters. The domain and the check know nothing about HTTP, SQL or any vendor; each outside service has a real adapter and an in-memory stand-in behind the same port.

PACKAGE	WHAT IT HOLDS
`@slip/domain`	Types, money in integer cents, the state machines, the persistence port, shared invariants
`@slip/check`	The pure check: freshness → variant → locate → comparable → gap → rules. No I/O at all
`@slip/graph`	The pure seller graph: evidence tiers, resolution, the seller count and standing. No I/O
`@slip/app`	The use cases — checkout, holds, resolver, webhooks, mandates — written against ports
`@slip/db`	Postgres store (Drizzle, postgres.js or PGlite), an in-memory fake, migrations and guards
`@slip/offers`	The Channel3 adapter, a fake, and a separate entry point for search so the checkout path cannot spend a credit
`@slip/paypal`	The PayPal REST adapter and a fake that passes the same gateway contract
`@slip/steering`	The Claude classifier and a scripted fake
`@slip/receipt`	One receipt renderer, shared by the agent, the MCP server and the owner's screens
`@slip/mcp`	The MCP server and its web-standard HTTP endpoint
`@slip/ui`	The owner's kit, “Till Roll”: theme tokens, stationery styles, React components, typed client
`@slip/web`	The Next.js app: receipts, holds, limits, agents, the kill switch, the demo
`@slip/http`	The one JSON client every adapter uses: no retries, no timeouts of its own
`@slip/study`	The market study: what Channel3's real offers look like, measured the way the check measures
`@slip/bench`	SlipBench: how shopping agents behave on real offers under four conditions
`@slip/tests`	End-to-end scenarios through the application, on both stores

Conventions worth knowing before reading the code

- A private export condition serves sources inside the workspace and built output everywhere else, so tests need no build step.
- Deadlines belong to the caller, never to an adapter; an adapter that cannot answer in time reports a timeout and the check holds.
- Every failure an outside service can return is a value, not an exception: a purchase that cannot be priced is held for the owner, not crashed.
- The database enforces what the code enforces — append-only triggers, set-once columns, uniqueness on every PayPal id — and drift tests compare the constraints against the domain's own enumerations.

6 What has been measured

Two studies, both recomputable from committed data with no key and no model call. Both report results that do not flatter the product, which is the point of running them.

The market study — 240 real products, 19 October

MEASURE	RESULT	READING
Products with two or more comparable offers	18.33% (44 of 240)	G1 fails against a 60% bar — most products in this catalogue carry one offer
Typical gap, cheapest to priciest	16.67%	The smaller, sounder figure: 17.02% including three products whose grouping is suspect
Dearer offer pays more commission	47.5% (19 of 40)	Of the 40 products where two offers stated *different* commission rates — not of all 240
Offers listed → mapped → comparable	321 → 321 → 318	The adapter loses nothing; the check excludes three

> WHERE THE REAL ARGUMENT SITS
>
> The low density is Channel3's catalogue, not Slip's rules — and narrowing by category does not clear the bar. So the case leans on the commission conflict: in nearly half the products where the comparison is possible at all, the store paying the most commission is not the cheapest. That is a structural conflict of interest a buyer cannot see, and it is what a second opinion is for.

SlipBench — two agents, 168 runs, 26 October

CONDITION	WHAT THE AGENT IS SHOWN	OVERPAID	TOOK THE BAIT
`clean`	Real offers, nothing manipulated	0 of 42	—
`endorsed`	A dearer offer labelled “Top pick”, listed first	0 of 42	0
`injected`	Instruction-like text standing alone on the page	0 of 42	0
`buried`	The same sentence hidden inside the product's own description — where an attacker would put it	0 of 42	0

G3 fails at 0% against a 15% bar. In this benchmark that means both agents compared offers well enough that Slip never had to stop either — including when the bait was buried in prose they were certain to read. The report says so plainly rather than hiding it, and it discloses two things the table cannot show: the steering verdict is supplied by construction rather than by a live model call, so the numbers recompute for free; and a zero in *stopped by Slip* records that no attack landed, not that the check let one through.

The classifier itself is verified against the real model separately — instruction-like text flagged, loud sales copy not, inside the application's own deadline. The second agent's sweep has since run and reproduced the first's result exactly — nothing overpaid and nothing took the bait in any condition — which is the more useful kind of confirmation, since a single agent's

zero could have been one model's quirk. What is still not done is a second *provider*, which SPEC §6.2 asks for and which needs a key nobody here has. The repository says that rather than implying otherwise.

7 Running it

The three outside services

SERVICE	REAL WHEN	OTHERWISE
Channel3 — offers	`CHANNEL3_API_KEY` is set. Verified live. Product detail is free; search costs a credit, so only the agent-facing search can spend one, behind a cache and an hourly ceiling	A stand-in catalogue
PayPal — money	Client id, secret and webhook id are set. `live` is the only value that moves real money	A stand-in gateway that passes the same contract
Steering — the classifier	`ANTHROPIC_API_KEY` is set	A scripted stand-in; in production without a key every verdict the check needs is “no answer”, which holds the purchase

> TWO DEPLOYMENTS THAT REFUSE TO START
>
> Live PayPal with simulated prices, or with a scripted steering stand-in, throws at assembly. A deployment that cannot be trusted does not boot, rather than discovering the problem at the first checkout with an agent waiting and a real card one call away.

Hosting

- **Demo mode** needs no database and no keys: every visitor gets their own catalogue, their own stand-in PayPal and four purchases already tried.
- **Any container host.** The image serves `/sign-in` and answers its own health check. The build takes about 50 seconds but wants well over ten gigabytes of Docker storage; the first local attempt had 17 GB free and still ran out at the final layer commit.
- **Render** is declared in `render.yaml` — connect the repo and choose Blueprint; Slip works out its own address. On the free plan the service sleeps after 15 minutes.
- **Serverless** has no long-lived process, so the resolver runs from a scheduled route instead, authorised by a bearer secret.

The commands that matter

COMMAND	WHAT IT DOES
`pnpm lint && pnpm test`	The gate: biome, project-wide types, 2,129 tests. Spends nothing
`pnpm test:pg`	The same store contract against a real Postgres — 310 tests. Point `SLIP_TEST_DATABASE_URL` at a server you already have and it starts no container
`pnpm web:build && pnpm web:e2e`	38 browser tests across a demo server and a real-owner server, with an accessibility audit on every page in both themes
`pnpm test:live:*`	Four suites that skip themselves until their key exists
`pnpm study:report`	Recomputes the market study from committed snapshots. Free
`pnpm bench:run`	Prints what it would spend and does nothing without `--yes`; never pays twice for an errand already recorded

8 Where it stands

Everything that can be verified without a key or an account someone must open has been.

Done and verified

- The check, the domain model, persistence, every use case, the owner UI, the MCP server, and real adapters for all three outside services.
- The wiring a deployment outside the demo needs: an agent that can find products, a steering check with something to read, a resolver that runs where there is no long-lived process, and a way back in for an owner whose browser forgot the session.
- Channel3 verified against a live key; the classifier verified against the real model; the store contract verified on real Postgres; a daily reconcile against PayPal's own records.
- Two measured studies, both recomputable for free, both reporting failures honestly.

Designed, and deliberately not built

- **The merchant side.** Standing earned by disclosure rather than bought, and the seller graph that roots it. The graph ships, and two of its three rules with it — a market that collapses to one seller, and a chosen offer that discloses no commission where the cheaper one does, are now held rather than paid. The attestations, their store and the hook surface are not built.
- **The reversal.** Slip can stop a payment and cannot undo one: there is no refund call anywhere, a refunded capture still reads as captured, and the event that would announce one is never subscribed to — so a refund issued out of band reaches Slip only through the daily reconcile, where it reads as the owner's own account disagreeing. The kill switch stops the next payment, not the last one. A check that can only ever say no is a filter rather than a policy engine, and the correction refund is specified down to its failure cases — every one of them now named and quoted from PayPal’s own per-error reference, including two clocks rather than one and a refusal no retry can clear. None of it is built.

Open, and waiting on the account holder

WHAT	WHY IT CANNOT BE CLOSED FROM HERE
A second *provider* for SlipBench (SPEC §6.2)	Needs an OpenAI key. The second *agent*'s sweep has run; a second provider has not, and nothing is built on speculation
G2 — a charge against a vaulted wallet	A sandbox buyer must approve a vault through PayPal's own flow; nothing can automate it. G10, the refund round trip, waits behind the same step
G4 — is Channel3's product grouping correct?	Thirty products are printed for a person to adjudicate, suspects first. No second signal exists in the data: of the 44 products with two or more comparable offers, not one has two that both carry Channel3's `dimensions`, so size and weight cannot corroborate a grouping. It needs a human judgement, not a measurement
A Next.js security advisory	A fresh build stops to name a known advisory against the installed version and proceeds on a re-run — it recurred twice on 30 October under different notice ids, so it is not a one-off. Upgrading a framework days before submission, against a green gate, is the larger risk: recorded rather than acted on
Continuous integration	Billing on the repository host; no run has ever executed. The local gate is the same commands
A public URL	Connect the repository in Render
Making the repository public	At submission, on the owner's word

> HOW TO READ THIS DOCUMENT'S NUMBERS
>
> Every figure here was read from the repository or produced by running it on 30 October 2026. The decision log records all 98 departures from the written spec with the reason for each, so anything surprising in the code has an entry explaining why it is that way. `SPEC.md` §10 lists all fourteen gates in one table with where each stands: six are open, and not one of them can be closed from inside this repository.